Overheads of Hardware vs. Software Virtualization

A quantitive analysis of KVM and QEMU TCG Memory subsystems

Milad Zarei - Taha Biklariyan - Nazanin Sharifi

OS Tracing - Dr. Azhari @ IUST

Semester 4042

Background

- Virtualization is widely-used in cloud computing, OS development, etc.
- Allows you to run multiple *guest operating systems* inside a *host operating system*.

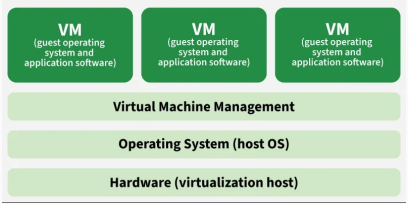


Figure 1: Virtualization

Background cont.

- **Hardware-Assisted Virtualization:**
  - Powered by KVM in the Linux kernel for virtualization on the same host architecture.
  - Processing is done on actual hardware via CPU Virtualization technologies (VT-x/AMD-V).
- **Emulation:**
  - For running OSes that run on entirely different CPU architectures!
  - Powered by QEMU's TCG (Tiny Code Generator).

Motivation: The Virtual Memory Challenge

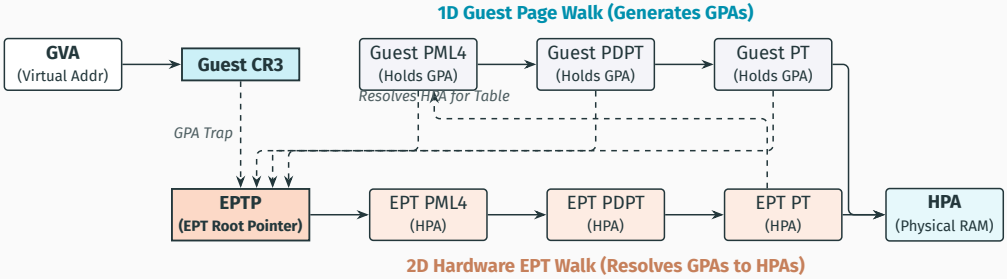
The Guest Memory Layering Illusion:

- Guest OS maps Guest Virtual Addresses (GVA) to Guest Physical Addresses (GPA).
- Host OS must map GPA to final Host Physical Addresses (HPA).

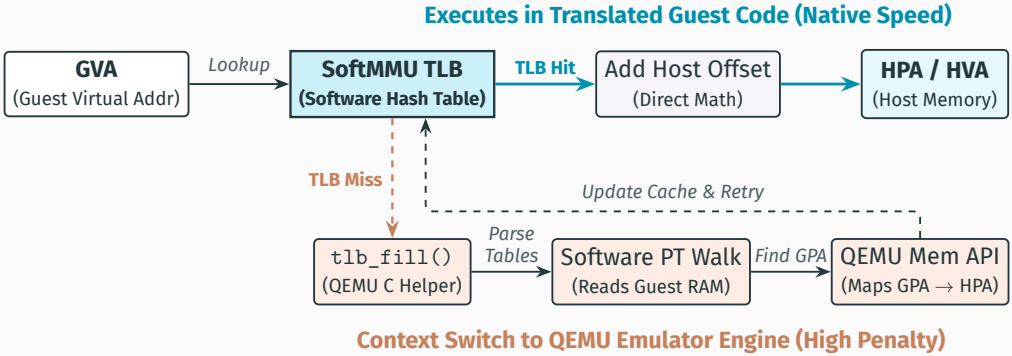
Two Architectural Solutions:

- **Hardware EPT (KVM):** The hardware MMU natively walks both page tables simultaneously in silicon. High-speed, but TLB misses are costly.
- **Software MMU (TCG):** QEMU maintains an inline software TLB cache. If it misses, it triggers a slow, multi-layered C helper routine loop.

EPT Overall Look



SoftMMU Overall Look



Motivation: Instruction Pipeline Divergence

Instruction Handling Disparity:

- **KVM Direct Execution:** Guest instructions execute natively on physical execution units at full hardware frequency.
- Overheads only trigger during trapping instructions (*VM-Exits*) like I/O or hypercalls.

TCG Binary Translation:

- Guest binary blocks are completely intercepted, compiled to Intermediate Representation (IR), and rewritten into Host Assembly.
- Code changes require synchronous, costly generation cache invalidations (*SMC Penalties*).

Project Goal: Quantifying the Virtualization Tax

- **The Main Objective:** Isolate and measure the exact architectural overheads of hardware-assisted memory management (KVM/EPT) versus software emulation (QEMU/SoftMMU).

- **The Telemetry Stack:** Utilize Linux profiling tools to expose cycle-level bottlenecks:
  - `perf stat` for hardware counters (cache misses, dTLB stalls).
  - `perf record` and `perf meme` to map execution hotspots.
  - **Flamegraphs** to visually pinpoint wasted CPU cycles in the virtualization stack.

- **Workload Strategy:** Deploy targeted benchmarks (Pointer Chaser, Matrix Multiplication, Self-Modifying Code) to explicitly stress memory latency, compute boundaries, and JIT translation penalties.

Experimental Setup: The Shielded Environment

- **Host Configuration & CPU Isolation:**
  - Dual-core CPU pinning and isolation via kernel command-line parameters
  - Hyper-threading and IRQ handling explicitly disabled for those cores to eliminate L1/L2 cache pollution.
  - *For further*

- **Guest VM Infrastructure:**
  - Orchestrating Debian (NoCloud images) via custom QEMU/KVM scripts.
  - **Mixed-Architecture Target:**
    - **x86_64:** Native KVM vs. TCG Emulation.
    - **ARM (AArch64):** Cross-Architecture TCG Emulation.

- **Telemetry Pipeline:**
  - Fully automated data collection communicating via SSH.
  - `perf` executed on host to preserve isolated guest execution.

Isolation details & scripts: `https://github.com/Tahabik/M-C-Analysis`

Workload Methodology: The Benchmarks

- **Pointer Chaser (Memory Latency Bound):**
  - Traverses a linked list with randomized and sequential memory allocations.
  - **Stress Profile:** Maximizes dTLB misses and forces constant cache eviction to evaluate the raw penalty of EPT page walks versus SoftMMU table parsing.

- **Self-Modifying Code (Translation Bound):**
  - Generates or alters instructions at runtime before executing them.
  - **Stress Profile:** Directly forces the invalidation of JIT-compiled translation blocks. Exposes the execution penalty of QEMU TCG code generation cache flushes.

- **Matrix Multiplication (Compute & Cache Bound):**
  - Performs dense linear algebra operations ($O(n^3)$ complexity).
  - **Stress Profile:** Exhibits high spatial and temporal data locality. Tests how well each subsystem handles streaming data across L1, L2, and L3 cache boundaries.

Benchmark 1 Analysis Roadmap: Pointer Chaser

- **Phase 1: Microarchitectural Profiling (`perf stat`)**
  - Baseline evaluation of Sequential vs. Random access patterns.
  - Hard metrics tracking: IPC, dTLB stalls, and LLC miss ratios.
- **Phase 2: Memory Characterization (`perf mem` & `report`)**
  - Mapping hit distributions across the cache hierarchy ($L1 \rightarrow L2 \rightarrow L3 \rightarrow$ DRAM).
  - Isolating the assembly-level stall points inside the dereference loop.
- **Phase 3: Macro-Execution Profile (Flamegraphs)**
  - Visualizing KVM kernel violation traps versus TCG engine execution.
  - Quantifying the exact ratio of JIT execution to `tlb_fill()` C helper overrides.

Phase 1: Microarchitectural Profiling (`perf stat`)

Core Metric Insights:

- **IPC Collapse:** Track the exact memory footprint size where IPC drops (e.g., when transitioning from L3 cache to DRAM).
- **The Translation Tax:** Contrast the massive instruction expansion in TCG versus KVM's near 1:1 execution mapping.
- **TLB vs. Cache Violations:** Pinpoint the size boundary where `dTLB-load-misses` spike for both environments.

Architectural Comparative Summary:

- **Sequential Access:** Hardware prefetchers effectively hide memory latency across both x86 and ARM in KVM.
- **Random Access (Stress):** TCG incurs high software TLB miss costs; KVM suffers from costly 2D hardware page walks.
- **Instruction Stream:** TCG shows significantly higher `L1-icache` and `iTLB` misses due to dynamic JIT code generation.

Phase 1: Microarchitectural Profiling: KVM vs. TCG (`perf stat`)

Pointer Chaser Telemetry at Extremes (L1d vs. DRAM Bound)

Metric	x86 KVM (Hardware)	ARM TCG (SoftMMU)	Multiplier / Insight
Baseline Latency (16KB Seq)	1.13 ns	3.81 ns	3.3x (SoftMMU Base Overhead)
Stress Latency (64MB Rand)	61.09 ns	75.92 ns	Gap narrows at physical DRAM limit
Total Instructions Executed	0.92 Billion	29.74 Billion	**32x Instruction Bloat** in TCG
L1 iCache Misses (64MB Rand)	1.4 Million	140.4 Million	**100x Thrashing** from JIT/C Helpers
dTLB Misses (64MB Rand)	38,000	482,000	Heavy software page-table parsing

Key Architectural Takeaways:

- **Instruction Expansion:** TCG generates massive instruction bloat to maintain software MMU hash tables compared to KVM's native 2D page walk.
- **Pipeline Pollution:** TCG dynamic translation completely destroys instruction locality, resulting in 100x more instruction cache misses.
- **The Memory Wall:** Both virtualization strategies hit severe bottlenecks at 16MB, indicating host L3 cache overflow.

Benchmark 1 Synthesis: The Master Telemetry Matrix

Telemetry Vector	Metric	Hardware (KVM)	Software (TCG)	Architectural Reality
Pipeline Profiling	Total Instructions	0.92 Billion	29.74 Billion	~**32x Bloat:** Native vs. SoftMMU hash math
Code Translation	iCache Misses *(64MB)*	1.4 Million	140.4 Million	~**100x Thrashing:** JIT context switching
Mem Translation	dTLB Misses *(64MB)*	38,000	482,000	**12.6x Rate:** EPT vs. SoftMMU table parsing
Latency Bounds	16KB → 64MB Rand	1.13 ns → 61.09 ns	3.81 ns → 75.92 ns	Gap narrows heavily at the physical DRAM wall
Latency Origins	`perf mem` Hotspot	`eventfd_poll`	`tlb_fill`	KVM stalls on I/O; TCG stalls on translation
Execution Trap	`perf record` Top	`qemu_main_loop` (86%)	JIT Code Cache (80%)	KVM is trapped polling; TCG is trapped compiling

- **Final Conclusion:** Hardware virtualization is bound by the host kernel's I/O multiplexing, whereas software emulation actively sabotages the host's physical L1/L2 caches via dynamic translation overheads.

Phase 2 Setup: Contextual Latency Profiling

The Tool: `perf mem`

- **Beyond Counting:** Shifts from counting raw misses to capturing the exact *latency* (cycles) and *source* (L1, L2, RAM) of specific memory loads.
- **High-Latency Filtration:** We configure the tracer to ignore fast L1 hits and strictly log memory accesses taking > 30 CPU cycles.

The Objective: Hunting the Anomaly

- **Hypothesis Testing:** If hardware EPT is truly faster than SoftMMU, the physical location of the latency will look fundamentally different.
- **The SoftMMU Hook:** We are actively hunting for a highly specific, severe latency signature in TCG that *does not originate from the guest code*.

Phase 2 Results: The Source of Latency (`perf mem`)

High-Latency Load Distribution (> 30 CPU Cycles)

Hardware EPT (KVM)

Bottleneck Symbol	Subsystem
`kvm_lapic_find...`	Interrupt Routing
`eventfd_poll`	Host Kernel I/O
`update_curr`	Host Scheduler
`gsi_handler`	QEMU Event Loop

Insight: *Hardware page walks are fast enough that the primary latency shifts to virtual interrupt handling and host context switches.*

Software MMU (TCG)

Bottleneck Symbol	Subsystem
`tlb_fill`	SoftMMU Fallback
`cpu_mmu_lookup`	SoftMMU Hash Table
`helper_le_ldq_mmu`	JIT Memory Helper
`address_space_...`	QEMU Mem API

The Anomaly: *The SoftMMU data structures themselves are evicted to physical RAM. Emulation stalls parsing its own page tables.*

Phase 3 Setup: Visualizing the Execution Stack

The Tool: Call Graphs (`perf record -g`)

- **High-Frequency Sampling:** Captures the host CPU's exact execution stack the moment a hardware cache miss is triggered.
- **Chain of Custody:** Reconstructs the exact chain of function calls (who called whom) leading directly to the pipeline stall.

The Objective: The Flamegraph Abstraction

- **Macro-Level Translation:** Converting millions of dense, textual stack traces into visual, proportional execution blocks.
- **Isolating the Virtualization Tax:** Visually separating cycles spent executing guest memory accesses from cycles burned maintaining the emulator's infrastructure.

Next: Mapping the real-world traces against these expectations.

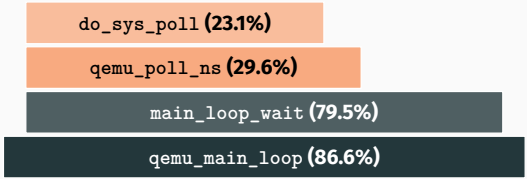
Phase 3: KVM Profile — The Polling Dominance

Host Stack Analysis:

- **The Event Loop Trap: 86.6%** of cache-miss samples originate from the core `qemu_main_loop`.
- **Syscall Bottleneck:** Drills straight down into `main_loop_wait` (79.5%) and `ppoll` (29.6%).
- **Kernel Tax:** 23.1% of stalls occur deep within the host kernel's `do_sys_poll`.

Takeaway: *Guest code execution is so fast that hardware stalls shift completely to host I/O multiplexing and kernel context switches.*

Dominant Execution Path:

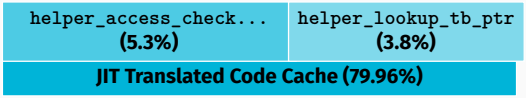


Phase 3: TCG Profile — The JIT Cache Pressure

Host Stack Analysis:

- **JIT Domination:** Nearly **80% (79.96%)** of all pipeline stalls happen directly inside the runtime JIT code buffer.
- **Translation Chaining:** Significant cycles are spent in `helper_lookup_tb_ptr` attempting to chain execution blocks together without exiting to the main loop.
- **Cache Thrashing:** High instruction-cache misses occur because the host CPU is constantly swinging between executing guest code and running SoftMMU lookup helpers.

Dominant Execution Path:



Takeaway: *Unlike KVM, TCG spends almost zero time polling. The host CPU is entirely bottle-necked compiling, executing, and resolving addresses for guest instructions.*

Concept Check: What is JIT (Just-In-Time) Compilation?

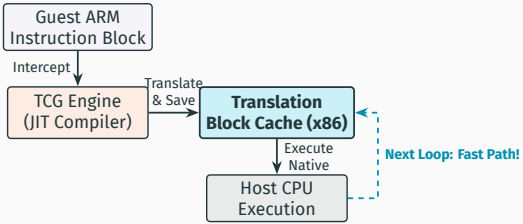
The Translation Problem:

- **Interpreter:** Translates code line-by-line, every single time. (Too slow).
- **Ahead-of-Time (AOT):** Pre-compiled for one specific CPU. (Inflexible).
- **JIT (The QEMU Way):** Translates chunks of code on the fly *and saves the result.*

Why it matters for our traces:

- Guest ARM instructions are compiled into Host x86 instructions *while* the VM runs.
- Once translated, the CPU executes the cached block at near-native speed.

The TCG JIT Pipeline:



Notice: If this cache is flushed, the CPU must fall back to step 1, destroying performance.

Introduction: The SMC Challenge in Virtual Environments

Understanding Self-Modifying Code (SMC)

- **The Concept:** SMC dynamically alters its own instructions during execution. This poses a unique challenge to hypervisors, which must safely track and execute these unexpected modifications without breaking guest isolation.

Hypervisor Approaches to SMC

- **KVM (Hardware-Assisted):**
  - Code executes directly on the native hardware pipeline.
  - Overheads remain minimal, primarily limited to standard hardware page faults or pipeline flushes natively handled by the host CPU.
- **QEMU TCG (Emulation / Binary Translation):**
  - Relies heavily on caching translated guest code in a *Translation Block (TB) cache*.
  - SMC forces TCG to aggressively and synchronously flush and recompile these blocks upon every modification.
  - **Result:** A severe performance bottleneck due to continuous JIT recompilation and cache invalidation.

Macroscopic Performance: Execution Time & Latency

Metric	x86 KVM (Hardware)	ARM TCG (Emulation)	Multiplier / Insight
Total Elapsed Time	0.014 s	1.369 s	$\sim 97\times$ Emulation Tax
Latency per Loop	141.69 ns	13,694.77 ns	$\sim 97\times$ Slower Execution
Throughput	7.058 MOps	0.073 MOps	Massive TCG Bottleneck
Total Instructions	171.6 Million	18.7 Billion	$\sim 108\times$ **Instruction Bloat**
L1 iCache Misses	4.32 Million	237.5 Million	$\sim 54\times$ **Cache Thrashing**

Key Architectural Takeaways:

- **The Emulation Tax:** TCG introduces a catastrophic $107\times$ latency penalty. Unlike KVM, which natively handles pipeline flushes in hardware, TCG must software-trap the modification.
- **Instruction Expansion:** Every time the code modifies itself, TCG must synchronously flush its Translation Block (TB) cache and invoke the JIT compiler, executing 18.7 billion instructions compared to KVM's 135 million.
- **Pipeline Pollution:** The constant swinging between executing guest code and recompiling IR entirely destroys instruction locality, destroying L1.

Microarchitectural Analysis: CPU Performance Counters i

Hardware Event	x86 KVM	ARM TCG	Microarchitectural Impact
Total Instructions	~ 171.6 M	~ 18.72 B	**JIT Recompilation:** Continuous IR-to-host translation upon every memory modification.
L1 dCache Misses	1.98 M	875.3 M	**Data Thrashing:** JIT and SoftMMU tables rapidly evict guest data from L1.
L1 iCache Misses	4.32 M	237.5 M	**Pipeline Pollution:** Constant TB invalidation destroys instruction spatial locality.
iTLB Load Misses	$138,238$	$900,037$	**TLB Pressure:** Rapid context switching between execution and translation strains host TLB.

Microarchitectural Analysis: CPU Performance Counters ii

Architectural Analysis:

- **Instruction vs. Execution:** In KVM, an SMC event is natively handled by the hardware pipeline. In TCG, altering a single instruction forces QEMU to drop execution, parse the modification, generate new TCG IR, and re-emit x86 host assembly, causing a $\sim 108\times$ instruction bloat.
- **Cache Destruction:** The catastrophic ~ 875 Million L1 data misses in TCG prove the host CPU is spending its time fetching QEMU's heavy internal translation mechanisms from slower DRAM tiers, entirely bottlenecking memory bandwidth.

Execution Hotspots: Memory Profiling (`perf mem`)

Pinpointing High-Latency Memory Overheads

Hardware Execution (KVM)

Bottleneck Symbol	Subsystem (Overhead)
`native_write_msr`	Hardware State (79.55%)
`vcpu_run`	VCPU Execution (11.36%)
`_copy_to_user`	Kernel Memory (Minor)

Insight: *The host CPU remains fully in execution mode. The memory footprint is entirely dedicated to native hardware-level state management.*

Software Emulation (TCG)

Bottleneck Symbol	Subsystem (Overhead)
`tcg_optimize`	JIT Compiler (34.20%)
`cpu_get_tb_cpu...`	TB Cache Mgmt (3.88%)
`arm_cpu_tlb_fill...`	SoftMMU (2.30%)
`tcg_gen_code`	JIT Engine (Minor)

The Anomaly: *QEMU spends its high-latency memory accesses trapped in its own mechanics—managing the software TLB and dynamically compiling blocks, rather than executing guest code.*

Introduction: The Predictable Workload (GEMM)

The Concept: Dense Mathematical Loops

- General Matrix Multiply (GEMM) is characterized by highly predictable, tight, nested loops performing dense floating-point arithmetic.

The Contrast: An Emulator's "Ideal" Scenario

- **Unlike Pointer Chasing:** Memory access is strictly strided and predictable, not random.
- **Unlike SMC:** The code is completely static. It executes millions of times without self-modification.
- **The Advantage:** This predictability allows QEMU TCG's Translation Block (TB) cache to finally shine, completely avoiding the catastrophic invalidation penalties seen in the SMC benchmark.

The Objective

- To isolate and measure the *baseline "emulation tax"*. The pure computational cost of mapping instructions between architectures.

Macroscopic Performance: The Persistent Gap

Metric	x86 KVM (Hardware)	ARM TCG (Emulation)	Multiplier / Insight
Total Elapsed Time	0.028 s	0.197 s	$\sim 7\times$ Emulation Tax
Throughput	1.194 GFLOPS	0.170 GFLOPS	Compute Bottleneck

Key Architectural Takeaways:

- **The "New" Insight ($\sim 7\times$ vs. $\sim 107\times$):** While a $7\times$ slowdown is significant, it is a massive improvement over the $\sim 107\times$ penalty observed in the SMC benchmark.
- **Caching Success:** This proves that when guest code is static and does not modify itself, TCG's Translation Block (TB) caching mechanism is highly effective. The emulator successfully avoids the catastrophic invalidation loops.
- **The Baseline Tax:** Even with perfect caching, the hypervisor cannot escape the fundamental cost of translating complex ARM floating-point instructions.

Microarchitectural Analysis: Instruction Mapping

Hardware Event	x86 KVM	ARM TCG	Microarchitectural Impact
Total Instructions	~ 297.6 M	~ 9.03 B	**Instruction Bloat ($\sim 30\times$):** ARM FPU to x86 translation requires complex mapping and soft-float logic.
L1 dCache Misses	19.0 M	54.5 M	**Pre-fetcher Victory (Only $\sim 2.8\times$):** Hardware prefetchers successfully anticipate strided array accesses.

Architectural Analysis:

- **The FPU Mapping Problem:** Even with perfectly cached execution, translating ARM floating-point/SIMD instructions to x86 is not $1:1$.
- **The Pre-fetcher Victory:** Unlike the random accesses in pointer chasing, matrix loops are highly predictable. The host CPU's hardware prefetchers can effectively anticipate these memory accesses even through emulation.